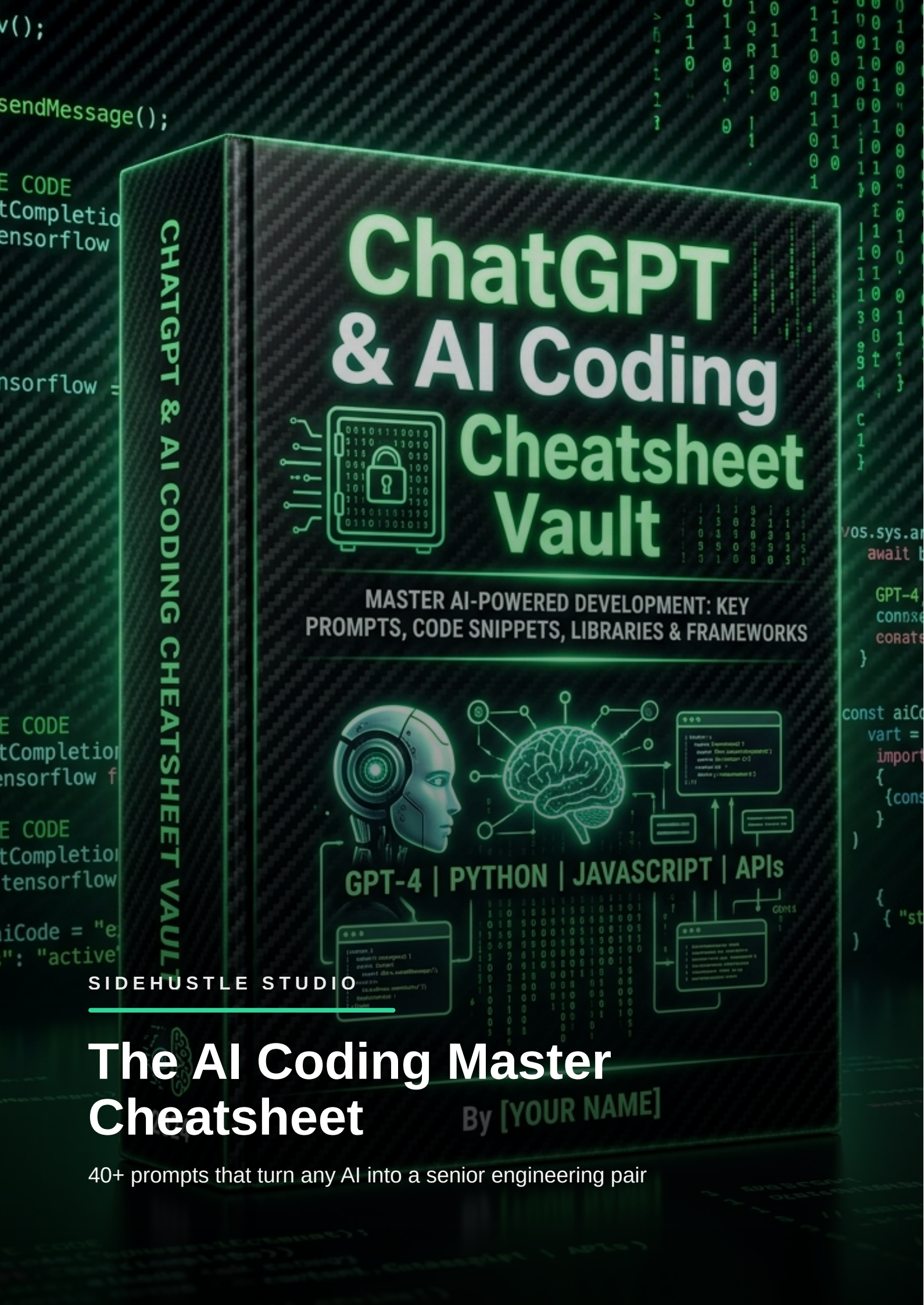




v());

sendMessage();

E CODE
tCompleti
ensorflow

ensorflow =

E CODE
tCompleti
ensorflow f

E CODE
tCompleti
ensorflow

iCode = "e
": "active"

vos.sys.a
await b

GPT-4
condse
corate

const aiCo
var =
import
{cons
}
}
{ "st
}

CHATGPT & AI CODING CHEATSHEET VAULT

ChatGPT & AI Coding Cheatsheet Vault



MASTER AI-POWERED DEVELOPMENT: KEY
PROMPTS, CODE SNIPPETS, LIBRARIES & FRAMEWORKS



GPT-4 | PYTHON | JAVASCRIPT | APIs

SIDEHUSTLE STUDIO

The AI Coding Master Cheatsheet

By [YOUR NAME]

40+ prompts that turn any AI into a senior engineering pair

The AI Coding Master Cheatsheet

40+ battle-tested prompts that turn ChatGPT, Claude & Gemini into a senior engineering pair — for shipping, debugging, refactoring, testing, and learning faster.

Paste your code where you see [PASTE CODE]. Every prompt sets a role and constraints so you get production-grade output, not toy answers.

How to prompt an AI for code (the 5 rules pros use)

1. **Give it the whole file, not a fragment.** Context prevents hallucinated APIs and wrong assumptions.
2. **State the stack and version.** "TypeScript 5.x, React 19, Node 22" stops it from writing outdated code.
3. **Ask for the diff, not the whole file,** on edits — faster to review, less to break.
4. **Demand it explain trade-offs,** not just output. "Give me 2 approaches and recommend one."
5. **Never trust, always verify.** Run it, test it, read it. AI is a fast junior with a confident voice.

SECTION 1 — Writing & Scaffolding

Prompt — Production-Ready Function

Role: Principal software engineer.
Task: Write a [LANGUAGE] function that [DESCRIBE].
Requirements: strong typing, input validation, error handling, no external deps unless necessary, and JSDoc/docstring. Include 2 usage examples. Explain any non-obvious decision in one line.

Prompt — Scaffold a Module

Task: Scaffold a [LANGUAGE/Framework] module for [FEATURE]. Give me the file structure, the public interface, and stubbed functions with types and doc comments. Keep it modular and testable. Don't implement bodies yet – I want to review the shape first.

Prompt — Convert Pseudocode to Real Code

Task: Turn this pseudocode/logic [PASTE] into clean, idiomatic [LANGUAGE]. Follow the standard style guide for the language. Point out any edge cases my logic missed.

Prompt — Boilerplate from a Spec

Task: Given this spec [PASTE], generate the boilerplate: types/interfaces, the main class/functions, and TODO-free stubs I can fill in. Use [Framework] conventions.

Prompt — Regex Builder (that you can read)

Task: Write a regex that matches [DESCRIBE]. Then: explain each part in plain English, give 3 matching and 3 non-matching examples, and warn me of any catastrophic-backtracking risk.

SECTION 2 — Debugging & Diagnosis

Prompt — Root-Cause Bug Hunt

Role: Senior debugging specialist.

Task: Given this error/stack trace [PASTE] and the relevant code [PASTE CODE], explain the root cause in 2 sentences, then give me the precise diff to fix it. Don't mask the symptom – fix the underlying issue. Note if similar bugs likely exist elsewhere.

Prompt — "It Works Locally, Not in Prod"

Task: This behaves differently in [ENV A] vs [ENV B]: [DESCRIBE]. List the 5 most likely causes ranked by probability (env vars, timing, versions, permissions, data), and the fastest way to confirm each.

Prompt — Rubber-Duck a Logic Error

Task: This function should [EXPECTED] but does [ACTUAL] for input [INPUT]: [PASTE CODE]. Trace the execution step by step with that input and tell me exactly where it diverges.

Prompt — Reproduce Then Fix

Task: Here's an intermittent bug: [DESCRIBE]. First write a minimal test that reliably reproduces it, then fix the code so the test passes. Show both.

Prompt — Decode a Cryptic Error

Task: Explain this error message in plain English: [PASTE]. What is it actually telling me, what are the 3 most common causes, and what should I check first?

SECTION 3 — Refactoring & Optimisation

Prompt — Refactor for Readability

Role: Staff engineer who values clarity.

Task: Refactor this code [PASTE] to be cleaner and more maintainable without changing behaviour. Reduce nesting, name things well, remove dead code. Show a before/after diff and list what you changed and why.

Prompt — Performance Optimisation

Task: Analyse this code [PASTE] for performance issues: unnecessary loops, $O(n^2)$ hotspots, repeated work, N+1 queries. Give me the optimised version and quantify the expected improvement. Don't sacrifice readability unless the gain is significant – say so if it's a trade-off.

Prompt — Reduce Complexity

Task: This function is too complex [PASTE]. Break it into smaller, single-responsibility functions with clear names. Preserve behaviour exactly. Keep the public interface the same.

Prompt — Modernise Legacy Code

Task: Update this legacy [LANGUAGE] code [PASTE] to modern idioms and syntax ([target version]). Replace deprecated APIs, use current best practices, and flag anything that changes behaviour so I can test it.

Prompt — DRY Out Duplication

Task: I have these similar blocks [PASTE]. Extract the shared logic into a well-named reusable function/component without over-abstracting. Show the refactor and the updated call sites.

SECTION 4 — Testing

Prompt — Full Unit Test Suite

Task: Write comprehensive [Jest/Vitest/pytest] unit tests for this code [PASTE]. Cover happy path, edge cases, error cases, and boundary conditions. Aim for meaningful branch coverage, not just line coverage. Use clear test names that describe behaviour.

Prompt — Find the Missing Test Cases

Task: Here's my code [PASTE] and my current tests [PASTE]. What cases am I NOT testing? List the gaps ranked by risk, then write tests for the top 5.

Prompt — Test Data & Mocks

Task: Generate realistic test fixtures and mocks for [FUNCTION/MODULE]. Include valid, invalid, empty, and malicious inputs. Keep them minimal but representative.

Prompt — Integration Test Plan

Task: Design an integration test plan for [FEATURE] that touches [COMPONENTS]. What should be tested end-to-end, what should be mocked, and what are the key failure scenarios?

SECTION 5 — Review & Security

Prompt — Senior Code Review

Role: Meticulous senior reviewer.
Task: Review this PR/code [PASTE]. Identify: bugs, security issues, race conditions, performance concerns, edge cases, and maintainability problems. For each, give severity, the concrete failure scenario, and a suggested fix. Rank by severity.

Prompt — Security Audit

Task: Audit this code [PASTE] for security vulnerabilities: injection, XSS, auth flaws, secret exposure, unsafe deserialization, and dependency risks. For each finding, show how it could be exploited and the fix. Assume it's going to production.

Prompt — Input Validation Hardening

Task: Add robust input validation and sanitisation to this endpoint/function [PASTE]. Validate types, ranges, and formats; reject or safely handle bad input; and never trust client data. Use [validation approach/library] if appropriate.

Prompt — Concurrency & Race Conditions

Task: Review this concurrent/async code [PASTE] for race conditions, deadlocks, and shared-state bugs. Explain any issue with a concrete interleaving that triggers it, and show the fix.

SECTION 6 — Architecture & Design

Prompt — Design a System

Role: Software architect.
Task: Design [SYSTEM/FEATURE] for [SCALE/CONSTRAINTS]. Give me: the high-level components, data model, key trade-offs, and 2 alternative approaches with a recommendation. Call out what would break at 10x scale.

Prompt — Choose Between Options

Task: I'm deciding between [A] and [B] for [PURPOSE]. Compare on: fit for my use case, complexity, performance, maintainability, ecosystem, and long-term risk. Recommend one for a project starting today and justify it.

Prompt — Data Model Review

Task: Review this schema/data model [PASTE] for [USE CASE]. Flag normalization issues, missing indexes, scaling risks, and relationships that will hurt later. Suggest improvements.

Prompt — API Design

Task: Design a clean REST/GraphQL API for [RESOURCE]. Give me endpoints/schema, request/response shapes, status codes, error format, versioning approach, and pagination. Follow current best practices.

SECTION 7 — Documentation & Learning

Prompt — Document This Code

Task: Write clear documentation for this code [PASTE]: what it does, params, returns, errors, and a usage example. Then write a plain-English summary a non-expert teammate could understand.

Prompt — Explain Like I'm the New Dev

Task: Explain what this code [PASTE] does, line by line where non-obvious, as if onboarding a capable dev who's new to this codebase. Highlight the "why," not just the "what."

Prompt — Write the README

Task: Write a README for this project [DESCRIBE/PASTE package.json]. Include: what it does, install, quick start, config, common commands, and troubleshooting. Concise and scannable.

Prompt — Learn a New Concept Through My Code

Task: I want to understand [CONCEPT, e.g. dependency injection]. Explain it using MY code [PASTE] as the example – show my current approach, then how the concept would change it, and when it's worth it.

Prompt — Commit Message & PR Description

Task: Given this diff [PASTE], write a clear conventional-commit message and a PR description covering what changed, why, and how to test. Keep it meaningful, not verbose.

SECTION 8 — Speed & Everyday Utilities

Prompt — One-Liner / CLI Helper

Task: Give me the exact [bash/PowerShell] command to [TASK]. Explain each flag briefly and warn me if any part is destructive or irreversible.

Prompt — Convert Between Languages

Task: Port this [SOURCE LANG] code [PASTE] to [TARGET LANG], keeping behaviour identical and using idiomatic patterns of the target. Note anything that doesn't translate cleanly.

Prompt — Explain a Dependency

Task: What does [PACKAGE] actually do, what are its main use cases, its lighter alternatives, and any red flags (bundle size, maintenance, security)? Should I add it or is there a built-in way?

Prompt — SQL Query Builder

Task: Write a SQL query that [DESCRIBE] against these tables [PASTE SCHEMA]. Optimise it, explain the joins, and flag any performance risk on large tables.

Prompt — Turn a Bug Report into a Fix Plan

Task: Here's a vague user bug report: [PASTE]. Translate it into: likely reproduction steps, the probable component at fault, and a prioritised investigation plan.

Prompt — Estimate the Work

Task: Break this feature [DESCRIBE] into concrete engineering tasks with rough effort (S/M/L), dependencies, and the riskiest unknown to de-risk first.

Prompt — Git Rescue

Task: I did [DESCRIBE GIT SITUATION] and need to [DESIRED OUTCOME] without losing work. Give me the exact safe git commands in order, explain what each does, and flag the point of no return.

The golden rule

AI writes code at the speed of thought and the accuracy of a confident junior. Your job shifts from typing to **reviewing**: read every line, run every path, test every edge. Used this way, these prompts easily save 10–20 hours a week. Used blindly, they ship bugs at scale. Be the senior in the loop.

© [YOUR NAME]. Licensed for personal and commercial use by the purchaser. Do not resell or redistribute this document as-is.